39. Show that if a game of nim begins with two piles containing the same number of stones, as long as this number is at least two, then the second player wins when both players follow optimal strategies.

40. Show that if a game of nim begins with two piles containing different numbers of stones, the first player wins when both players follow optimal strategies.

41. How many children does the root of the game tree for checkers have? How many grandchildren does it have?

42. How many children does the root of the game tree for nim have and how many grandchildren does it have if the starting position is

a) piles with four and five stones, respectively.

b) piles with two, three, and four stones, respectively.

c) piles with one, two, three, and four stones, respectively.

d) piles with two, two, three, three, and five stones, respectively.

43. Draw the game tree for the game of tic-tac-toe for the levels corresponding to the first two moves. Assign the value of the evaluation function mentioned in the text that assigns to a position the number of files containing no Os minus the number of files containing no Xs as the value of each vertex at this level and compute the value of the tree for vertices as if the evaluation function gave the correct values for these vertices.

44. Use pseudocode to describe an algorithm for determining the value of a game tree when both players follow a minmax strategy.

11.3 Tree Traversal

11.3.1 Introduction

Links

Ordered rooted trees are often used to store information. We need procedures for visiting each vertex of an ordered rooted tree to access data. We will describe several important algorithms for visiting all the vertices of an ordered rooted tree. Ordered rooted trees can also be used to represent various types of expressions, such as arithmetic expressions involving numbers, variables, and operations. The different listings of the vertices of ordered rooted trees used to represent expressions are useful in the evaluation of these expressions.

11.3.2 Universal Address Systems

Procedures for traversing all vertices of an ordered rooted tree rely on the orderings of children. In ordered rooted trees, the children of an internal vertex are shown from left to right in the drawings representing these directed graphs.

We will describe one way we can totally order the vertices of an ordered rooted tree. To produce this ordering, we must first label all the vertices. We do this recursively:

1. Label the root with the integer 0. Then label its k children (at level 1) from left to right with $1, 2, 3, \ldots, k$.
2. For each vertex v at level n with label A, label its k_v children, as they are drawn from left to right, with $A.1, A.2, \ldots, A.k_v$.

Following this procedure, a vertex v at level n, for $n \geq 1$, is labeled $x_1.x_2.\ldots.x_n$, where the unique path from the root to v goes through the x_1st vertex at level 1, the x_2nd vertex at level 2, and so on. This labeling is called the **universal address system** of the ordered rooted tree.

We can totally order the vertices using the lexicographic ordering of their labels in the universal address system. The vertex labeled $x_1.x_2.\ldots.x_n$ is less than the vertex labeled $y_1.y_2.\ldots.y_m$ if there is an i, $0 \leq i \leq n$, with $x_1 = y_1, x_2 = y_2, \ldots, x_{i-1} = y_{i-1}$, and $x_i < y_i$; or if $n < m$ and $x_i = y_i$ for $i = 1, 2, \ldots, n$.

EXAMPLE 1

Extra Examples

We display the labelings of the universal address system next to the vertices in the ordered rooted tree shown in Figure 1. The lexicographic ordering of the labelings is

$$0 < 1 < 1.1 < 1.2 < 1.3 < 2 < 3 < 3.1 < 3.1.1 < 3.1.2 < 3.1.2.1 < 3.1.2.2 < 3.1.2.3 < 3.1.2.4 < 3.1.3 < 3.2 < 4 < 4.1 < 5 < 5.1 < 5.1.1 < 5.2 < 5.3$$

◀

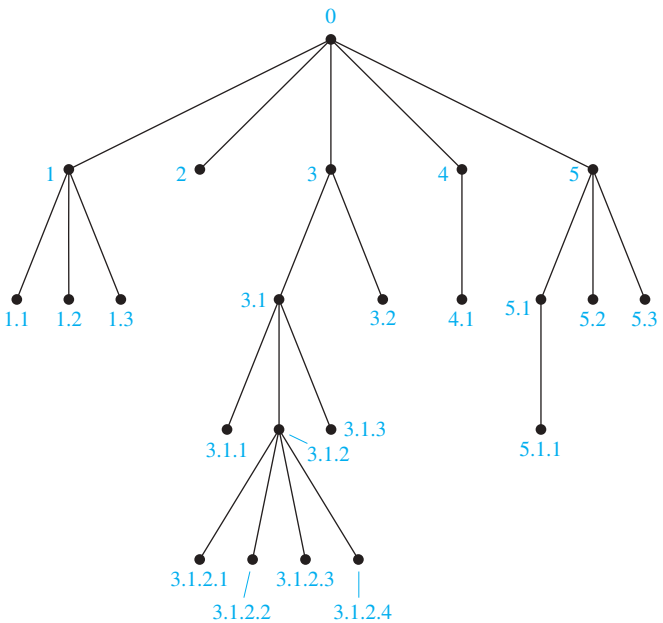


FIGURE 1 **The universal address system of an ordered rooted tree.**

11.3.3 Traversal Algorithms

Procedures for systematically visiting every vertex of an ordered rooted tree are called **traversal algorithms**. We will describe three of the most commonly used such algorithms, **preorder traversal**, **inorder traversal**, and **postorder traversal**. Each of these algorithms can be defined recursively. We first define preorder traversal.

Definition 1

Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *preorder traversal* of T. Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are the subtrees at r from left to right in T. The *preorder traversal* begins by visiting r. It continues by traversing T_1 in preorder, then T_2 in preorder, and so on, until T_n is traversed in preorder.

The reader should verify that the preorder traversal of an ordered rooted tree gives the same ordering of the vertices as the ordering obtained using a universal address system. Figure 2 indicates how a preorder traversal is carried out.

Example 2 illustrates preorder traversal.

EXAMPLE 2 In which order does a preorder traversal visit the vertices in the ordered rooted tree T shown in Figure 3?

Extra Examples

Solution: The steps of the preorder traversal of T are shown in Figure 4. We traverse T in preorder by first listing the root a, followed by the preorder list of the subtree with root b, the preorder list of the subtree with root c (which is just c) and the preorder list of the subtree with root d.

The preorder list of the subtree with root b begins by listing b, then the vertices of the subtree with root e in preorder, and then the subtree with root f in preorder (which is just f). The preorder list of the subtree with root d begins by listing d, followed by the preorder list of

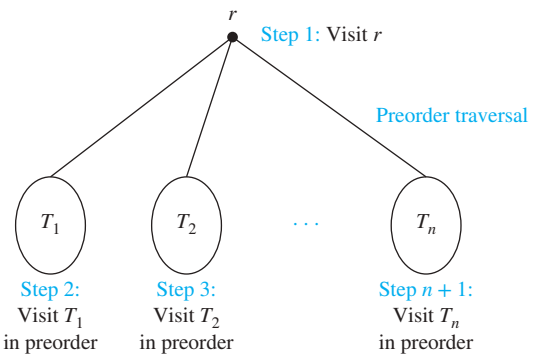


FIGURE 2 **Preorder traversal.**

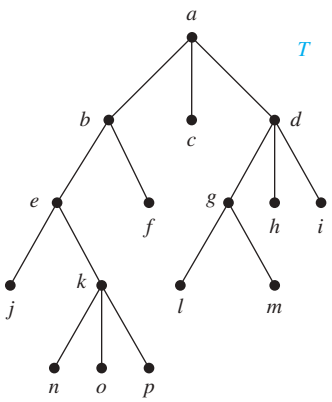


FIGURE 3 **The ordered rooted tree *T*.**

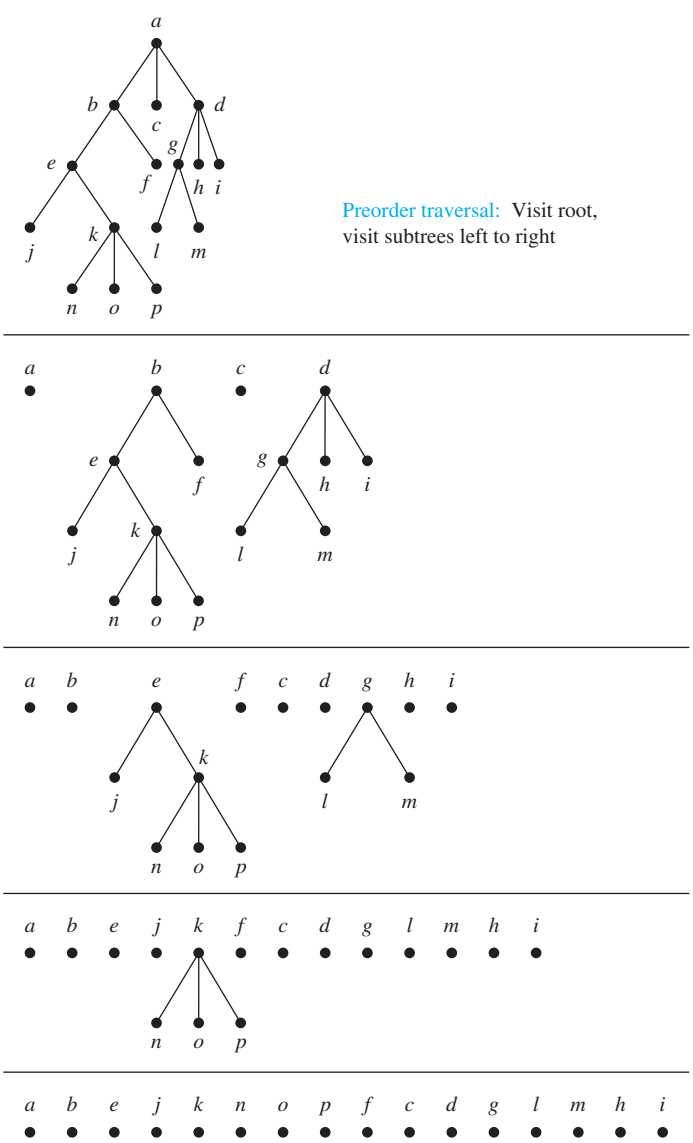


FIGURE 4 **The preorder traversal of *T*.**

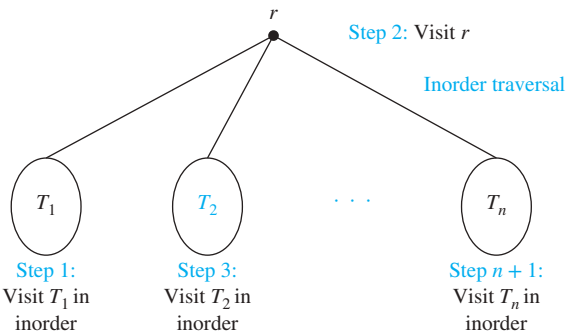


FIGURE 5 **Inorder traversal.**

the subtree with root g, followed by the subtree with root h (which is just h), followed by the subtree with root i (which is just i).

The preorder list of the subtree with root e begins by listing e, followed by the preorder listing of the subtree with root j (which is just j), followed by the preorder listing of the subtree with root k. The preorder listing of the subtree with root g is g followed by l, followed by m. The preorder listing of the subtree with root k is k, n, o, p. Consequently, the preorder traversal of T is $a, b, e, j, k, n, o, p, f, c, d, g, l, m, h, i$. ◀

We will now define inorder traversal.

Definition 2

Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *inorder traversal* of T. Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are the subtrees at r from left to right. The *inorder traversal* begins by traversing T_1 in inorder, then visiting r. It continues by traversing T_2 in inorder, then T_3 in inorder, $\ldots$, and finally T_n in inorder.

Figure 5 indicates how inorder traversal is carried out. Example 3 illustrates how inorder traversal is carried out for a particular tree.

EXAMPLE 3 In which order does an inorder traversal visit the vertices of the ordered rooted tree T in Figure 3?

Extra Examples

Solution: The steps of the inorder traversal of the ordered rooted tree T are shown in Figure 6. The inorder traversal begins with an inorder traversal of the subtree with root b, the root a, the inorder listing of the subtree with root c, which is just c, and the inorder listing of the subtree with root d.

The inorder listing of the subtree with root b begins with the inorder listing of the subtree with root e, the root b, and f. The inorder listing of the subtree with root d begins with the inorder listing of the subtree with root g, followed by the root d, followed by h, followed by i.

The inorder listing of the subtree with root e is j, followed by the root e, followed by the inorder listing of the subtree with root k. The inorder listing of the subtree with root g is l, g, m. The inorder listing of the subtree with root k is n, k, o, p. Consequently, the inorder listing of the ordered rooted tree is $j, e, n, k, o, p, b, f, a, c, l, g, m, d, h, i$. ◀

We now define postorder traversal.

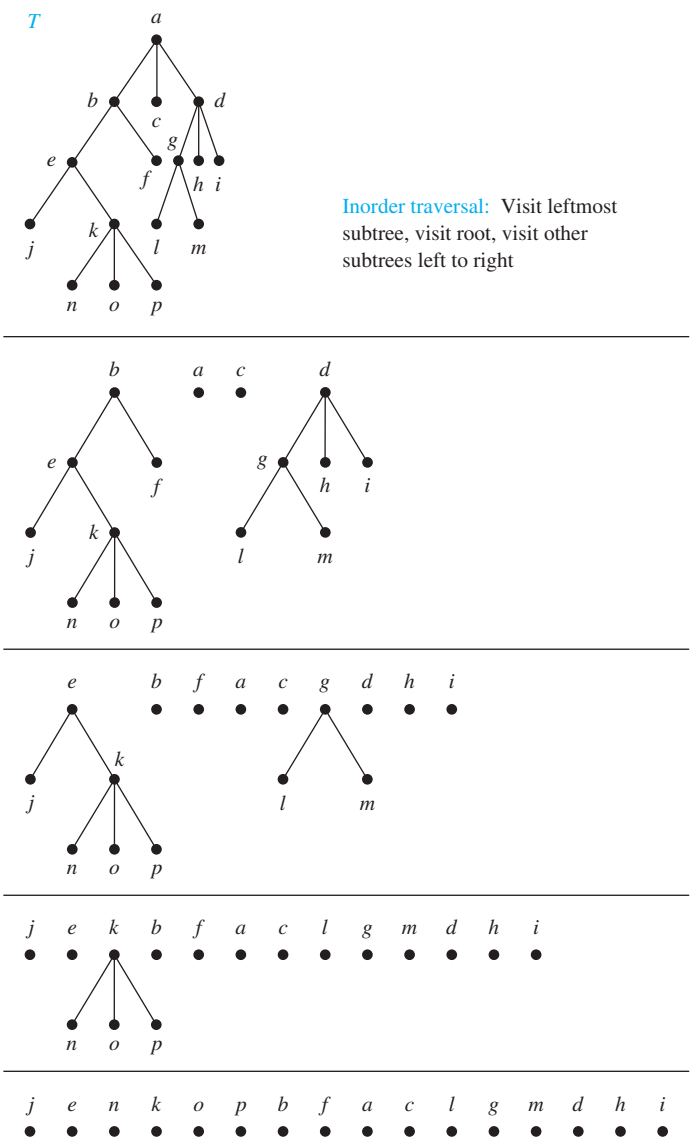


FIGURE 6 **The inorder traversal of *T*.**

Definition 3 Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *postorder traversal* of T. Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are the subtrees at r from left to right. The *postorder traversal* begins by traversing T_1 in postorder, then T_2 in postorder, $\ldots$, then T_n in postorder, and ends by visiting r.

Figure 7 illustrates how postorder traversal is done. Example 4 illustrates how postorder traversal works.

EXAMPLE 4 In which order does a postorder traversal visit the vertices of the ordered rooted tree T shown in Figure 3?

Extra Examples

Solution: The steps of the postorder traversal of the ordered rooted tree T are shown in Figure 8. The postorder traversal begins with the postorder traversal of the subtree with root b, the

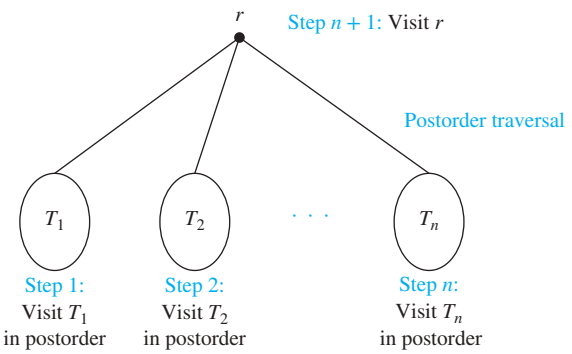


FIGURE 7 **Postorder traversal.**

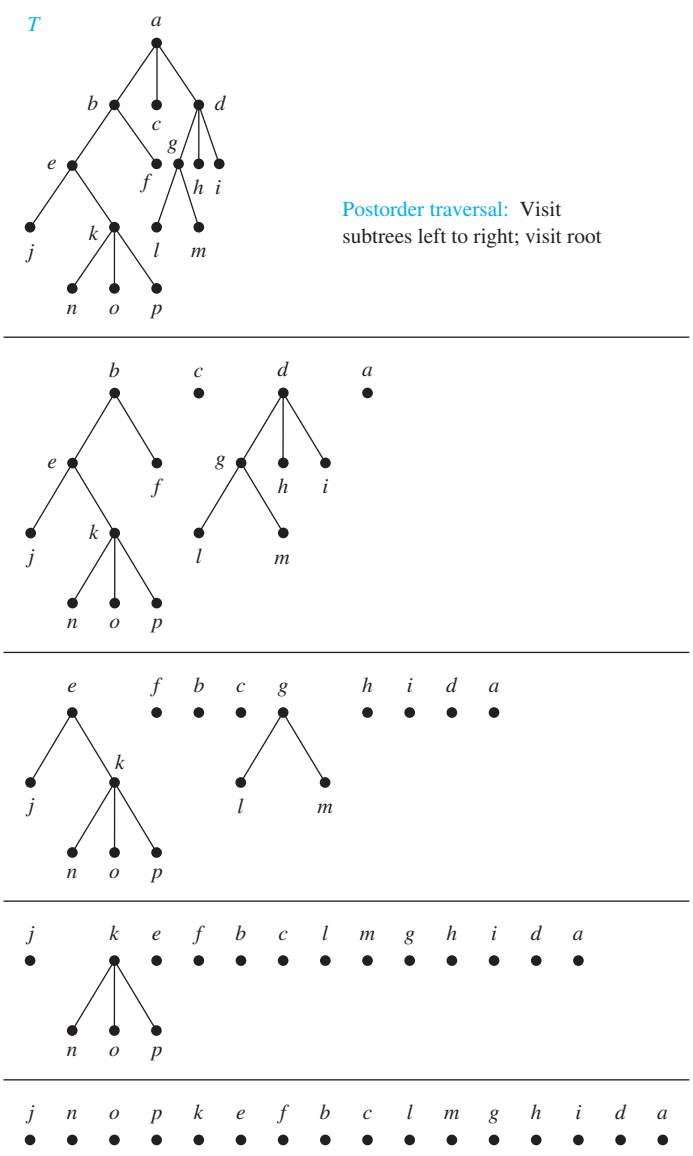


FIGURE 8 **The postorder traversal of *T*.**

postorder traversal of the subtree with root c, which is just c, the postorder traversal of the subtree with root d, followed by the root a.

The postorder traversal of the subtree with root b begins with the postorder traversal of the subtree with root e, followed by f, followed by the root b. The postorder traversal of the rooted tree with root d begins with the postorder traversal of the subtree with root g, followed by h, followed by i, followed by the root d.

The postorder traversal of the subtree with root e begins with j, followed by the postorder traversal of the subtree with root k, followed by the root e. The postorder traversal of the subtree with root g is l, m, g. The postorder traversal of the subtree with root k is n, o, p, k. Therefore, the postorder traversal of T is j, n, o, p, k, e, f, b, c, l, m, g, h, i, d, a. ◀

There are easy ways to list the vertices of an ordered rooted tree in preorder, inorder, and postorder. To do this, first draw a curve around the ordered rooted tree starting at the root, moving along the edges, as shown in the example in Figure 9. We can list the vertices in preorder by listing each vertex the first time this curve passes it. We can list the vertices in inorder by listing a leaf the first time the curve passes it and listing each internal vertex the second time the curve passes it. We can list the vertices in postorder by listing a vertex the last time it is passed on the way back up to its parent. When this is done in the rooted tree in Figure 9, it follows that the preorder traversal gives a, b, d, h, e, i, j, c, f, g, k, the inorder traversal gives h, d, b, i, e, j, a, f, c, k, g; and the postorder traversal gives h, d, i, j, e, b, f, k, g, c, a.

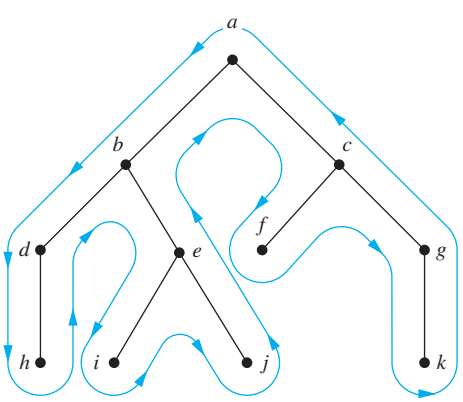


FIGURE 9 A shortcut for traversing an ordered rooted tree in preorder, inorder, and postorder.

Algorithms for traversing ordered rooted trees in preorder, inorder, or postorder are most easily expressed recursively.

ALGORITHM 1 Preorder Traversal.

procedure *preorder*(T: ordered rooted tree)
r := root of T
list r
for each child c of r from left to right
  $T(c)$:= subtree with c as its root
  preorder($T(c)$)

ALGORITHM 2 Inorder Traversal.

procedure *inorder*(T: ordered rooted tree)
r := root of T
if r is a leaf **then** list r
else
  l := first child of r from left to right
  $T(l)$:= subtree with l as its root
  inorder($T(l)$)
  list r
  for each child c of r except for l from left to right
    $T(c)$:= subtree with c as its root
    inorder($T(c)$)

ALGORITHM 3 Postorder Traversal.

procedure *postorder*(T: ordered rooted tree)
r := root of T
for each child c of r from left to right
  $T(c)$:= subtree with c as its root
  postorder($T(c)$)
list r

Both the preorder traversal and the postorder traversal encode the structure of an ordered rooted tree when the number of children of each vertex is specified. That is, an ordered rooted tree is uniquely determined when we specify a list of vertices generated by a preorder traversal or by a postorder traversal of the tree, together with the number of children of each vertex (see Exercises 26 and 27). In particular, both a preorder traversal and a postorder traversal encode the structure of a full ordered m-ary tree. However, when the number of children of vertices is not specified, neither a preorder traversal nor a postorder traversal encodes the structure of an ordered rooted tree (see Exercises 28 and 29).

Uses of Inorder, Preorder, and Postorder Traversals Tree traversals have many applications and they play a key role in the implementation of many algorithms. Binary ordered trees can be used to represent well-formed expressions of objects and operations. Preorder, postorder, and inorder traversals of these trees yield the prefix, postfix, and infix notations of expressions, which can then be used in applications. Before we turn our attention to that use of tree traversals, we will provide useful guidance about the practical use of tree traversals.

When efficiency is not an issue, the vertices of a tree can be visited in any order, so long as each node is visited exactly once. However, for other applications we may need to visit vertices in an order that preserves a specific relationship. Also, when efficiency is important, the most efficient traversal method for that application should be used. A general principle for deciding the traversal to use is to explore the vertices of interest as soon as possible. Preorder traversal is the best choice for applications where internal vertices must be explored before leaves. Preorder traversals are also used to make copies of binary search trees.

An interesting digression is to note that preorder traversal has ancient roots. According to Knuth [Kn98], when a king, duke, or earl dies, his title passes to his first son, then to the descendants of this son; when none of these are alive, it passes to the second son, and then his descendants, and so on. (In more modern times, daughters have also been included in this

order.) So, a preorder traversal of the vertices of the relevant family tree produces the order of succession to the throne, once deceased members are removed.

Postorder traversal is the best choice for applications where leaves need to be explored before internal vertices. Postorder explores leaves before internal vertices, so it is the best choice for deleting a tree because the vertices below the root of a subtree can be removed before the root of the subtree. Topological sorting is an example of an algorithm that can be efficiently implemented using postorder traversal. An inorder traversal of a binary search tree, discussed in Section 11.2, visits the vertices in ascending order of their key values. Such a traversal creates a sorted list of the data in a binary tree.

11.3.4 Infix, Prefix, and Postfix Notation

We can represent complicated expressions, such as compound propositions, combinations of sets, and arithmetic expressions using ordered rooted trees. For instance, consider the representation of an arithmetic expression involving the operators $+$ (addition), $-$ (subtraction), $*$ (multiplication), $/$ (division), and $\uparrow$ (exponentiation). We will use parentheses to indicate the order of the operations. An ordered rooted tree can be used to represent such expressions, where the internal vertices represent operations, and the leaves represent the variables or numbers. Each operation operates on its left and right subtrees (in that order).

EXAMPLE 5 What is the ordered rooted tree that represents the expression $((x + y) \uparrow 2) + ((x - 4)/3)$?

Solution: The binary tree for this expression can be built from the bottom up. First, a subtree for the expression $x + y$ is constructed. Then this is incorporated as part of the larger subtree representing $(x + y) \uparrow 2$. Also, a subtree for $x - 4$ is constructed, and then this is incorporated into a subtree representing $(x - 4)/3$. Finally the subtrees representing $(x + y) \uparrow 2$ and $(x - 4)/3$ are combined to form the ordered rooted tree representing $((x + y) \uparrow 2) + ((x - 4)/3)$. These steps are shown in Figure 10. ◀

An inorder traversal of the binary tree representing an expression produces the original expression with the elements and operations in the same order as they originally occurred, except for unary operations, which instead immediately follow their operands. For instance, inorder traversals of the binary trees in Figure 11, which represent the expressions $(x + y)/(x + 3)$, $(x + (y/x)) + 3$, and $x + (y/(x + 3))$, all lead to the infix expression $x + y/x + 3$. To make such expressions unambiguous it is necessary to include parentheses in the inorder traversal whenever we encounter an operation. The fully parenthesized expression obtained in this way is said to be in **infix form**.

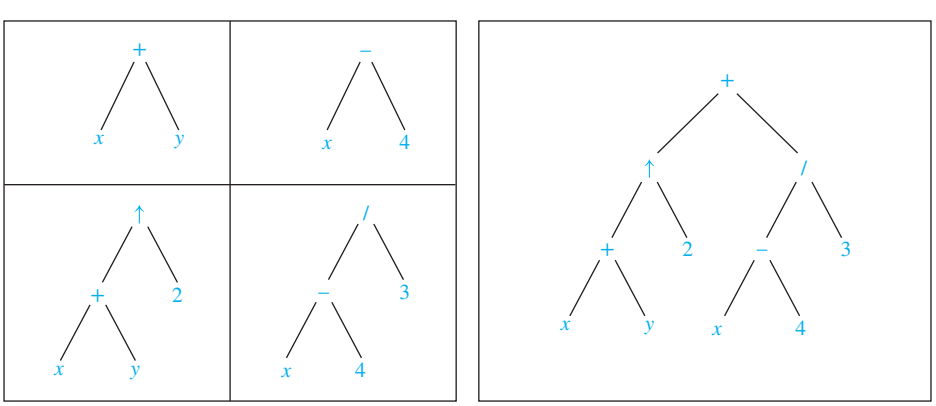


FIGURE 10 **A binary tree representing $((x + y) \uparrow 2) + ((x - 4)/3)$.**

We obtain the **prefix form** of an expression when we traverse its rooted tree in preorder. Expressions written in prefix form are said to be in **Polish notation**, which is named after the Polish logician Jan Łukasiewicz. An expression in prefix notation (where each operation has a specified number of operands), is unambiguous, so no parentheses are needed in such an expression. The verification of this is left as an exercise for the reader.

EXAMPLE 6 What is the prefix form for $((x + y) \uparrow 2) + ((x - 4)/3)$?

Solution: We obtain the prefix form for this expression by traversing the binary tree that represents it in preorder, shown in Figure 10. This produces $+ \uparrow + x\ y\ 2\ /\ -\ x\ 4\ 3$. ◀

In the prefix form of an expression, a binary operator, such as $+$, precedes its two operands. Hence, we can evaluate an expression in prefix form by working from right to left. When we encounter an operator, we perform the corresponding operation with the two operands immediately to the right of this operand. Also, whenever an operation is performed, we consider the result a new operand.

EXAMPLE 7 What is the value of the prefix expression $+ - * 2\ 3\ 5 / \uparrow 2\ 3\ 4$?

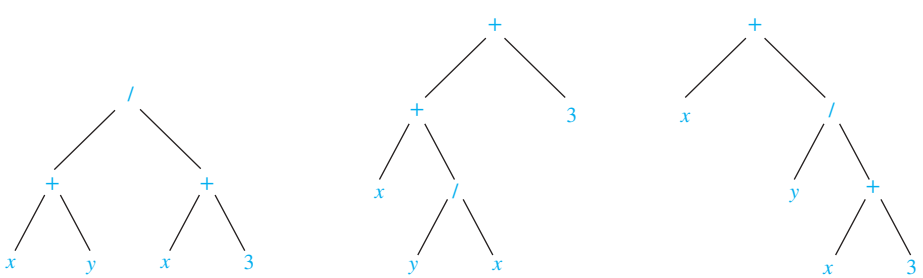


FIGURE 11 **Rooted trees representing $(x + y)/(x + 3)$, $(x + (y/x)) + 3$, and $x + (y/(x + 3))$.**

Solution: The steps used to evaluate this expression by working right to left, and performing operations using the operands on the right, are shown in Figure 12. The value of this expression is 3. ◀

Links

Reverse polish notation was first proposed in 1954 by Burks, Warren, and Wright.

We obtain the **postfix form** of an expression by traversing its binary tree in postorder. Expressions written in postfix form are said to be in **reverse Polish notation**. Expressions in reverse Polish notation are unambiguous, so parentheses are not needed. The verification of this is left to the reader. Reverse polish notation was extensively used in electronic calculators in the 1970s and 1980s.

EXAMPLE 8 What is the postfix form of the expression $((x + y) \uparrow 2) + ((x - 4)/3)$?

Solution: The postfix form of the expression is obtained by carrying out a postorder traversal of the binary tree for this expression, shown in Figure 10. This produces the postfix expression: $x\ y + 2 \uparrow x\ 4 - 3\ / +$. ◀

In the postfix form of an expression, a binary operator follows its two operands. So, to evaluate an expression from its postfix form, work from left to right, carrying out operations

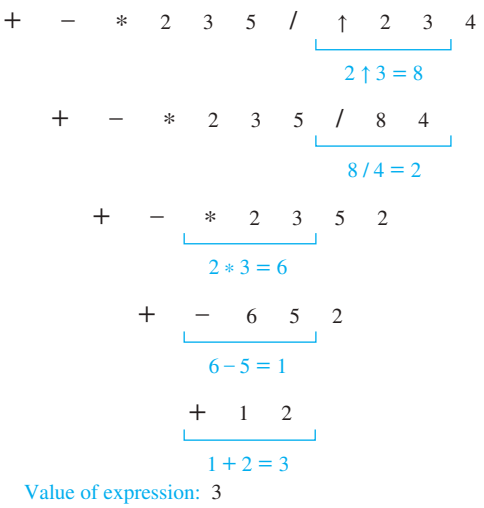


FIGURE 12 Evaluating a prefix expression.

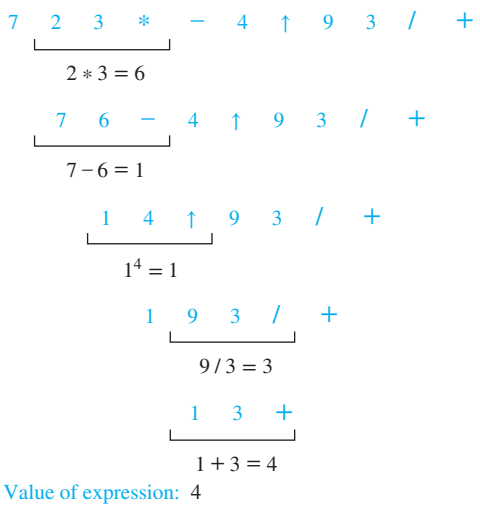


FIGURE 13 Evaluating a postfix expression.

whenever an operator follows two operands. After an operation is carried out, the result of this operation becomes a new operand.

EXAMPLE 9 What is the value of the postfix expression $7\ 2\ 3\ *\ -4\ \uparrow\ 9\ 3/+$?

Solution: The steps used to evaluate this expression by starting at the left and carrying out operations when two operands are followed by an operator are shown in Figure 13. The value of this expression is 4. ◀

Rooted trees can be used to represent other types of expressions, such as those representing compound propositions and combinations of sets. In these examples unary operators, such as the negation of a proposition, occur. To represent such operators and their operands, a vertex representing the operator and a child of this vertex representing the operand are used.

EXAMPLE 10 Find the ordered rooted tree representing the compound proposition $(\neg(p \wedge q)) \leftrightarrow (\neg p \vee \neg q)$. Then use this rooted tree to find the prefix, postfix, and infix forms of this expression.

Links

©UtCon Collection/Alamy Stock Photo

JAN ŁUKASIEWICZ (1878–1956) Jan Łukasiewicz was born into a Polish-speaking family in Lvov. At that time Lvov was part of Austria, but it is now in the Ukraine. His father was a captain in the Austrian army. Łukasiewicz became interested in mathematics while in high school. He studied mathematics and philosophy at the University of Lvov at both the undergraduate and graduate levels. After completing his doctoral work he became a lecturer there, and in 1911 he was appointed to a professorship. When the University of Warsaw was reopened as a Polish university in 1915, Łukasiewicz accepted an invitation to join the faculty. In 1919 he served as the Polish Minister of Education. He returned to the position of professor at Warsaw University, where he remained from 1920 to 1939, serving as rector of the university twice.

Łukasiewicz was one of the cofounders of the influential Warsaw School of Logic. He published his innovative text, *Elements of Mathematical Logic*, in 1928. With his influence, mathematical logic was made a required course for mathematics and science undergraduates in Poland. His lectures were considered excellent, even attracting students of the humanities.

Łukasiewicz and his wife experienced great suffering during World War II, which he documented in a posthumously published autobiography. After the war they lived in exile in Belgium. Fortunately, in 1949 he was offered a position at the Royal Irish Academy in Dublin.

Łukasiewicz worked on mathematical logic throughout his career. His work on a three-valued logic was an important contribution to the subject. Nevertheless, he is best known in the mathematical and computer science communities for his introduction of parenthesis-free notation, now called Polish notation.

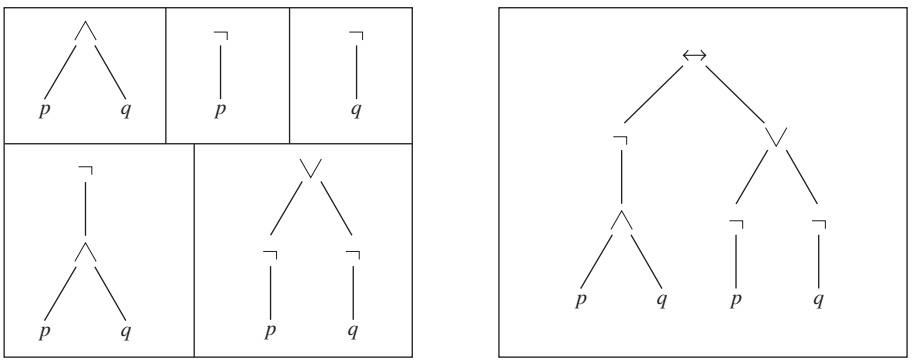


FIGURE 14 **Constructing the rooted tree for a compound proposition.**

Extra Examples

Solution: The rooted tree for this compound proposition is constructed from the bottom up. First, subtrees for $\neg p$ and $\neg q$ are formed (where $\neg$ is considered a unary operator). Also, a subtree for $p \wedge q$ is formed. Then subtrees for $\neg(p \wedge q)$ and $(\neg p) \vee (\neg q)$ are constructed. Finally, these two subtrees are used to form the final rooted tree. The steps of this procedure are shown in Figure 14.

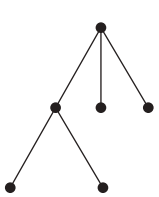
The prefix, postfix, and infix forms of this expression are found by traversing this rooted tree in preorder, postorder, and inorder (including parentheses), respectively. These traversals give $\leftrightarrow \neg \wedge pq \vee \neg p \neg q$, $pq \wedge \neg p \neg q \neg \vee \leftrightarrow$, and $(\neg(p \wedge q)) \leftrightarrow ((\neg p) \vee (\neg q))$, respectively. ◀

Because prefix and postfix expressions are unambiguous and because they can be evaluated easily without scanning back and forth, they are used extensively in computer science. Such expressions are especially useful in the construction of compilers.

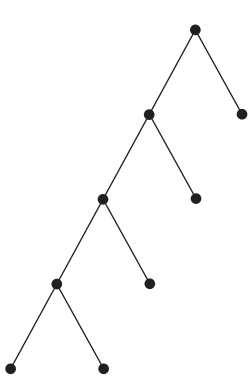
Exercises

In Exercises 1–3 construct the universal address system for the given ordered rooted tree. Then use this to order its vertices using the lexicographic order of their labels.

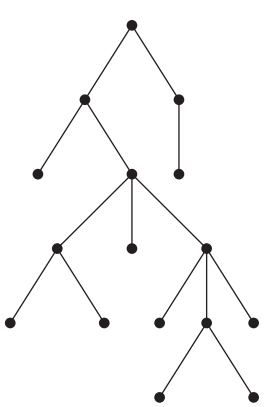
1.



2.



3.



4. Suppose that the address of the vertex v in the ordered rooted tree T is 3.4.5.2.4.

a) At what level is v?
b) What is the address of the parent of v?
c) What is the least number of siblings v can have?
d) What is the smallest possible number of vertices in T if v has this address?
e) Find the other addresses that must occur.

5. Suppose that the vertex with the largest address in an ordered rooted tree T has address 2.3.4.3.1. Is it possible to determine the number of vertices in T?

6. Can the leaves of an ordered rooted tree have the following list of universal addresses? If so, construct such an ordered rooted tree.

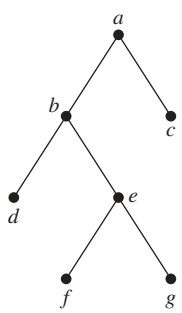
a) 1.1.1, 1.1.2, 1.2, 2.1.1.1, 2.1.2, 2.1.3, 2.2, 3.1.1, 3.1.2.1, 3.1.2.2, 3.2

b) 1.1, 1.2.1, 1.2.2, 1.2.3, 2.1, 2.2.1, 2.3.1, 2.3.2, 2.4.2.1, 2.4.2.2, 3.1, 3.2.1, 3.2.2

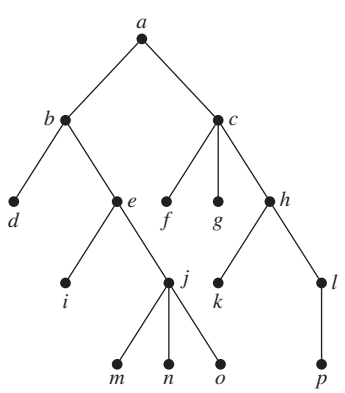
c) 1.1, 1.2.1, 1.2.2, 1.2.2.1, 1.3, 1.4, 2, 3.1, 3.2, 4.1.1.1

In Exercises 7–9 determine the order in which a preorder traversal visits the vertices of the given ordered rooted tree.

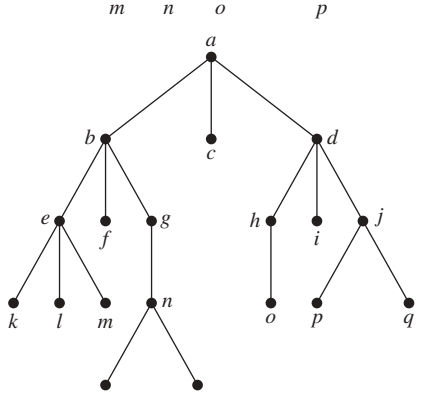
7.



8.



9.



10. In which order are the vertices of the ordered rooted tree in Exercise 7 visited using an inorder traversal?

11. In which order are the vertices of the ordered rooted tree in Exercise 8 visited using an inorder traversal?

12. In which order are the vertices of the ordered rooted tree in Exercise 9 visited using an inorder traversal?

13. In which order are the vertices of the ordered rooted tree in Exercise 7 visited using a postorder traversal?

14. In which order are the vertices of the ordered rooted tree in Exercise 8 visited using a postorder traversal?

15. In which order are the vertices of the ordered rooted tree in Exercise 9 visited using a postorder traversal?

16. **a)** Represent the expression $((x+2) \uparrow 3) * (y-(3+x)) - 5$ using a binary tree.

Write this expression in

b) prefix notation.

c) postfix notation.

d) infix notation.

17. **a)** Represent the expressions $(x+xy)+(x/y)$ and $x+((xy+x)/y)$ using binary trees.

Write these expressions in

b) prefix notation.

c) postfix notation.

d) infix notation.

18. **a)** Represent the compound propositions $\neg(p \wedge q) \leftrightarrow (\neg p \vee \neg q)$ and $(\neg p \wedge (q \leftrightarrow \neg p)) \vee \neg q$ using ordered rooted trees.

Write these expressions in

b) prefix notation.

c) postfix notation.

d) infix notation.

19. **a)** Represent $(A \cap B) - (A \cup (B - A))$ using an ordered rooted tree.

Write this expression in

b) prefix notation.

c) postfix notation.

d) infix notation.

***20.** In how many ways can the string $\neg p \wedge q \leftrightarrow \neg p \vee \neg q$ be fully parenthesized to yield an infix expression?

***21.** In how many ways can the string $A \cap B - A \cap B - A$ be fully parenthesized to yield an infix expression?

22. Draw the ordered rooted tree corresponding to each of these arithmetic expressions written in prefix notation. Then write each expression using infix notation.

a) $+ * + - 5\ 3\ 2\ 1\ 4$

b) $\uparrow + 2\ 3 - 5\ 1$

c) $* / 9\ 3 + * 2\ 4 - 7\ 6$

23. What is the value of each of these prefix expressions?

a) $- * 2 / 8\ 4\ 3$

b) $\uparrow - \ * 3\ 3 * 4\ 2\ 5$

c) $+ - \uparrow 3\ 2 \uparrow 2\ 3 / 6 - 4\ 2$

d) $* +3 + 3 \uparrow 3 + 3\ 3\ 3$

24. What is the value of each of these postfix expressions?

a) $5\ 2\ 1 - - 3\ 1\ 4 + + *$

b) $9\ 3 / 5 + 7\ 2 - *$

c) $3\ 2 * 2 \uparrow 5\ 3 - 8\ 4 / * -$

25. Construct the ordered rooted tree whose preorder traversal is *a, b, f, c, g, h, i, d, e, j, k, l*, where *a* has four children, *c* has three children, *j* has two children, *b* and *e* have one child each, and all other vertices are leaves.

***26.** Show that an ordered rooted tree is uniquely determined when a list of vertices generated by a preorder traversal of the tree and the number of children of each vertex are specified.